BridgeLabz

CASE STUDY DOCUMENT

Online Code Collaboration Platform

(Live Editor)

CodeSync

Code Together. Build Faster. Ship Smarter.

Version 1.0 | 2026

1. Synopsis

CodeSync is a full-stack Online Code Collaboration Platform that enables developers to write, execute, review, and version-control code in real time with teammates — all from a browser-based live editor, inspired by platforms like Replit and GitHub Codespaces. The system supports three primary roles: Guests (unauthenticated visitors who can browse public projects), Developers (registered users who create and collaborate on projects), and Administrators — each with a purpose-built interface.

Developers can create projects in any supported programming language, manage a multi-file project tree, open a browser-based code editor with full syntax highlighting, start or join live collaboration sessions with real-time cursor sharing and co-editing, execute code in sandboxed containers, view output and errors inline, create version snapshots, compare diffs between versions, restore previous states, add inline code review comments, and manage project member access.

CodeSync is architected as a microservices system modelled on the EShoppingZone reference pattern, with independent services for authentication/user management, project/repository management, file/editor management, real-time collaboration session management, code execution via sandboxed containers, version/snapshot control, code review comments, and notifications — all integrated through a Spring MVC website controller layer with WebSocket support for real-time collaboration.

2. Detailed Requirements

2.1 General Requirements

- Guests can browse public projects, view code files, and search users without logging in.
- Users must register or log in (email/password or GitHub/Google OAuth) to create or collaborate on projects.
- Role-based access: Guest, Developer (project member/owner), and Admin.
- All users can update their profile, username, avatar, bio, and password.
- In-app and email notifications are dispatched for session invites, comments, mentions, and snapshot events.
- Admin panel provides full user, project, execution, and session management with analytics.

2.2 Guest Requirements

- Browse and search public projects by language, name, or owner username.
- View public project file trees and read code files in read-only mode.
- View project descriptions, star counts, fork counts, and contributor lists.
- Access the registration and login pages.

2.3 Developer Requirements

- Register and log in via email/password or GitHub/Google OAuth.
- Create new projects with a name, description, programming language, and visibility (Public or Private).

- Create files and folder structures within a project; rename, move, and delete files.
- Edit code in the browser-based live editor with syntax highlighting, auto-indent, and bracket matching.
- Run code directly from the editor in a sandboxed container; provide custom stdin input; view stdout and stderr.
- Start a live collaboration session on any project file; share a session link or password-protect it.
- Join an active collaboration session; co-edit in real time with cursor presence indicators.
- See all active participants' cursors and selections highlighted in distinct colours.
- Create a snapshot (commit) of any file with a commit message to save the current state.
- View the version history of a file; see who created each snapshot and when.
- Compare any two snapshots with a diff view highlighting added/removed lines.
- Restore a file to any previous snapshot state.
- Create named branches for parallel development within a project.
- Tag snapshots with release labels (e.g., v1.0.0).
- Fork any public project to create a personal copy for experimentation.
- Star projects to bookmark them for later reference.
- Add inline code review comments on specific lines of any file.
- Reply to existing comments in a threaded format.
- Resolve or unresolve comments to track code review progress.
- Invite specific users or generate a share link to add members to a private project.
- Kick participants from an active collaboration session.
- Receive in-app notifications for: session invites, comment replies, @mentions, new snapshots by collaborators.

2.4 Admin Requirements

- View and manage all user accounts: suspend, reactivate, or permanently delete.
- View and moderate all projects; delete any project for policy violations.
- View all active collaboration sessions; terminate any session.
- View all code execution jobs; cancel any running job.
- Access platform analytics: total users, projects, executions, active sessions, code runs by language.
- Manage the set of supported programming languages and their sandbox images.
- Send platform-wide broadcast notifications to all or targeted users.
- View audit logs of all significant platform actions.

2.5 Code Execution Requirements

- Execution runs in isolated Docker containers (sandboxes) with configurable CPU, memory, and time limits.
- Supported languages include: Python, Java, JavaScript (Node.js), C, C++, Go, Rust, Ruby, TypeScript, PHP, Kotlin, Swift, and R.

- Each execution job records: language, source code, stdin, stdout, stderr, exit code, execution time (ms), and memory used (KB).
- Jobs are queued and processed asynchronously; status transitions: QUEUED → RUNNING → COMPLETED / FAILED / TIMED_OUT / CANCELLED.
- A WebSocket channel delivers real-time stdout streaming during execution for long-running programs.
- Sandbox resource limits: max 10 seconds execution, max 256MB memory, no network access, no filesystem write outside /tmp.

2.6 Collaboration Requirements

- Collaboration sessions are uniquely identified by a UUID session ID shared with invited participants.
- Each session is bound to a specific project file; multiple sessions can be active across different files of the same project.
- Operational Transformation (OT) or CRDT algorithm ensures consistency of concurrent edits across all participants.
- Cursor positions (line, column) are broadcast over WebSocket to all active participants in real time.
- Session owner can set a maximum participant limit and optionally require a password to join.
- Inactive sessions (no participant activity for 30 minutes) are automatically ended by a scheduled job.

2.7 Version Control Requirements

- A Snapshot captures the full content of a file at a point in time, with a SHA-256 content hash for integrity verification.
- Each snapshot references its parent snapshot ID, forming a linked chain — the file's history DAG.
- Branches are named pointers to snapshot chains; a project always has a default 'main' branch.
- Diff computation uses the Myers diff algorithm to produce line-by-line change annotations.
- Restoring a snapshot creates a new snapshot (with the old content) rather than modifying history.

2.8 Notification Requirements

- Notifications are dispatched for: session invite, participant joined/left, comment added, comment mention (@username), snapshot created, project forked.
- Each notification carries actorId, recipientId, relatedId (sessionId/commentId/snapshotId), and a deep-link URL.
- Unread notification badge count is shown in the navigation bar in real time via WebSocket push.
- Bulk broadcast is available for admin platform-wide announcements.
- Email notifications are sent for direct session invites and comment @mentions.

3. Use Case Diagram

The diagram below captures all actors and use cases within the CodeSync platform — spanning authentication, project management, file editing, live collaboration, code execution, version control, code review, notifications, and administrative management.

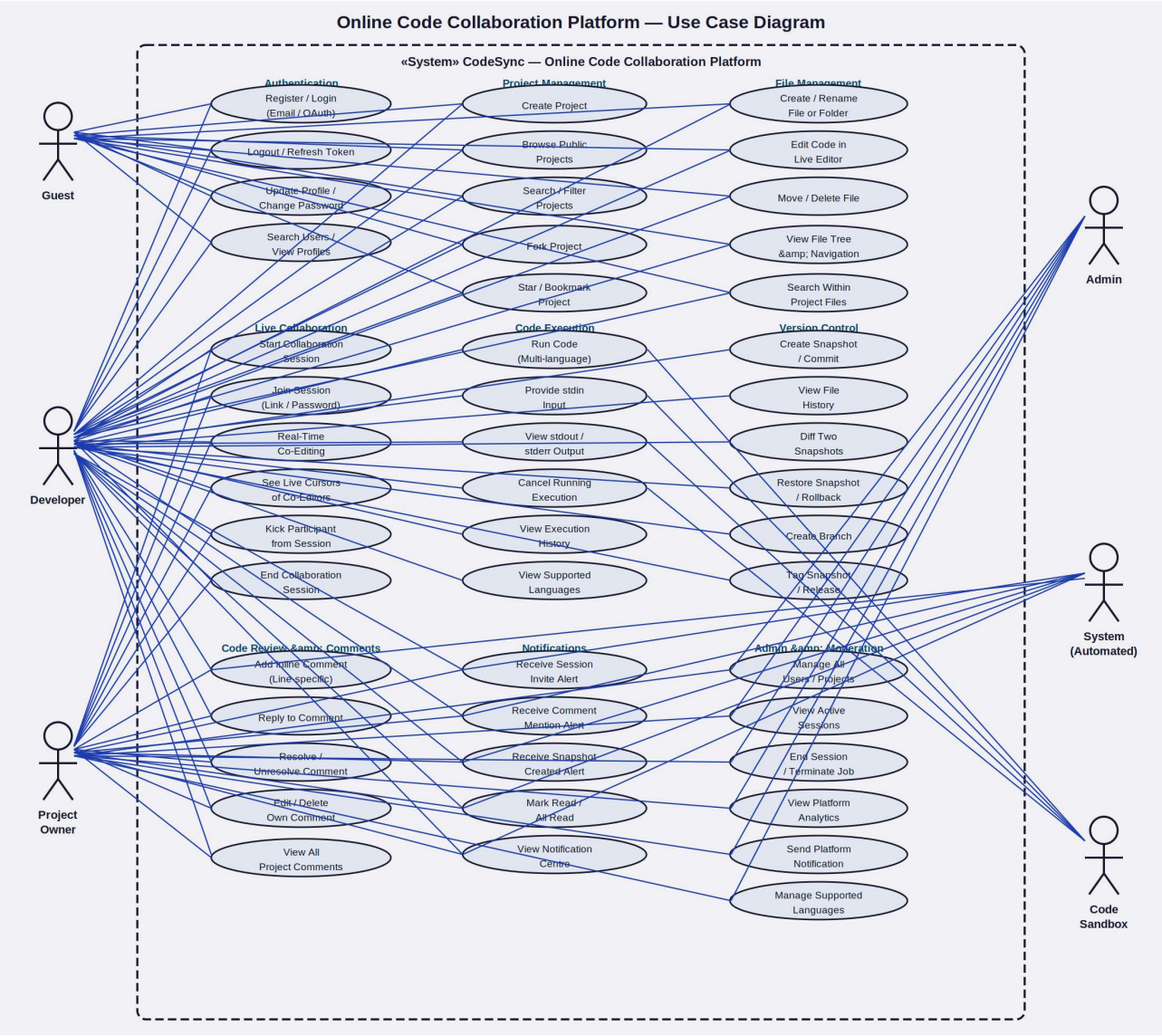

Figure 1: CodeSync Online Code Collaboration Platform — Complete Use Case Diagram

3.1 Actors

Actor	Description
Guest	Unauthenticated visitor who can browse public projects and view code in read-only mode
Developer	Registered user who creates projects, writes code, runs programs, and collaborates in real time
Project Owner	Developer who created the project and has full control over members, sessions, and settings
Admin	Platform administrator who manages users, projects, sessions, executions, and analytics
System (Automated)	Automated component handling inactive session cleanup, snapshot notifications, and scheduled jobs
Code Sandbox	Isolated Docker container environment that safely executes submitted code

3.2 Use Case Summary

Use Case	Actor(s)	Description
Register / Login	Guest, Developer	Create account or authenticate via email or GitHub/Google OAuth
Update Profile	Developer	Change username, avatar, bio, or password
Search Users / View Profile	Guest, Developer	Find developers by username and view their public projects
Create Project	Developer	New project with name, language, description, and visibility setting
Browse Public Projects	Guest, Developer	Explore public projects filterable by language or name
Search / Filter Projects	Guest, Developer	Find projects by name, owner, or programming language
Fork Project	Developer	Clone a public project into the developer's own account
Star / Bookmark Project	Developer	Bookmark a project for quick access; increment the project's star count
Create / Rename File	Developer	Add a new file or folder to the project tree; rename existing ones
Edit Code in Live Editor	Developer	Write code with syntax highlighting, auto-indent, and bracket matching
Move / Delete File	Developer	Reorganise or remove files from the project's directory structure

Use Case	Actor(s)	Description
View File Tree	Guest, Developer	Navigate the hierarchical folder and file structure of a project
Search Within Project	Developer	Find text or code patterns across all files in a project
Start Collaboration Session	Developer	Open a live co-editing session on a specific file with a shareable link
Join Session	Developer	Enter an active collaboration session via link or password
Real-Time Co-Editing	Developer	Simultaneously edit the same file with all session participants
See Live Cursors	Developer	View colour-coded cursor and selection positions of all co-editors
Kick Participant	Project Owner	Remove a specific participant from an active session
End Collaboration Session	Project Owner	Terminate the session, preventing new edits from being applied
Run Code	Developer, Code Sandbox	Submit code to a sandboxed container and receive output
Provide stdin Input	Developer	Supply custom standard input data for interactive program execution
View stdout / stderr	Developer	Read the program output and error streams after or during execution
Cancel Execution	Developer	Terminate a running execution job before it completes
View Execution History	Developer	Browse past execution jobs with status, output, and timing details
View Supported Languages	Guest, Developer	See the list of available programming languages with version info
Create Snapshot	Developer	Save the current state of a file with a descriptive commit message
View File History	Developer	Browse the chronological list of snapshots for a file
Diff Two Snapshots	Developer	Compare any two snapshots with line-by-line difference highlighting
Restore Snapshot	Developer	Roll back a file to a previous snapshot state
Create Branch	Developer	Create a named branch for parallel development on a project
Tag Snapshot	Developer	Attach a semantic version label (e.g., v1.0) to a specific snapshot

Use Case	Actor(s)	Description
Add Inline Comment	Developer	Attach a code review comment to a specific line in a file
Reply to Comment	Developer	Post a threaded reply under an existing comment
Resolve / Unresolve Comment	Developer, Project Owner	Mark a comment as addressed or reopen it for further discussion
Edit / Delete Comment	Developer	Modify or remove own comments from the review thread
View All Project Comments	Developer	See all open, resolved, and inline comments for a project
Receive Notifications	Developer	Get in-app alerts for session invites, comments, and snapshots
Mark Read / All Read	Developer	Dismiss individual or all unread notifications
Manage All Users	Admin	View, suspend, reactivate, or delete any user account
View Active Sessions	Admin	Monitor all currently active collaboration sessions platform-wide
End Session / Cancel Job	Admin	Force-terminate any collaboration session or running execution
View Platform Analytics	Admin	Access total users, projects, runs by language, and session metrics
Manage Supported Languages	Admin	Add, update, or disable programming languages and their sandbox images
Send Platform Notification	Admin	Broadcast an alert to all or a targeted group of users
End Inactive Session	System	Auto-terminate sessions with no participant activity for 30 minutes
Execute Code in Sandbox	Code Sandbox	Run submitted code in isolated container; return stdout/stderr/exitCode

4. Class Diagrams — All Services

CodeSync follows the same layered microservices architecture as the EShoppingZone reference system. Each service comprises five layers: Entity/POJO (domain model), Repository Interface (data access), Service Interface (business contract), ServiceImpl (business logic), and REST Resource (API exposure). The ten class diagrams below detail every service in the platform.

4.1 Auth / User-Service

The Auth/User-Service is the security gateway for the entire platform. It manages registration, login, JWT token lifecycle, OAuth2 provider integration (GitHub/Google), profile management, username/email changes, and account deactivation. The User entity carries a provider field to distinguish local accounts from OAuth-linked accounts, and a role field enabling role-based API gating across all services.

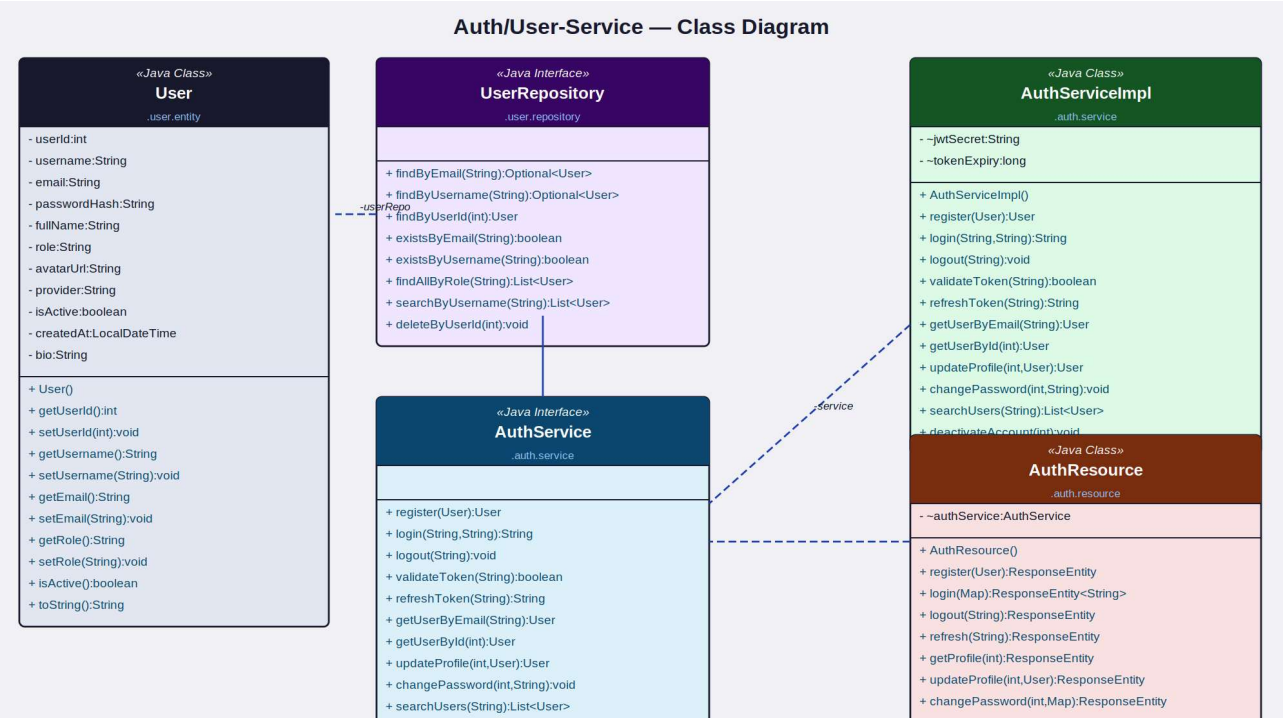

Figure 2: Auth/User-Service — Class Diagram

Component	Type	Responsibility
User	Java Class (Entity)	Stores userId, username, email, passwordHash, fullName, role (DEVELOPER/ADMIN), avatarUrl, provider (LOCAL/GITHUB/GOOGLE), isActive, createdAt, bio
UserRepository	Java Interface	findByEmail(), findByUsername(), findByUserId(), existsByEmail(), existsByUsername(), findAllByRole(), searchByUsername(), deleteByUserId()
AuthServiceImpl	Java Class	Implements register, login, logout, JWT generation/validation, OAuth2 handling, profile update, password change, user search, account deactivation
AuthService	Java Interface	Declares register(), login(), logout(), validateToken(), refreshToken(), getUserByEmail(),

Component	Type	Responsibility
		getUserById(), updateProfile(), changePassword(), searchUsers(), deactivateAccount()
AuthResource	Java Class (REST)	Exposes /auth/register, /auth/login, /auth/logout, /auth/refresh, /auth/profile (GET/PUT), /auth/password, /auth/search, /auth/deactivate

4.2 Project / Repository-Service

The Project/Repository-Service manages the top-level code project container. Projects have a visibility setting (PUBLIC or PRIVATE) controlling discoverability. The service tracks star counts and fork counts as denormalised fields for efficient sorting and display on discovery feeds. Forking creates a new project owned by the requesting user with the source project's file structure copied over via the File-Service.

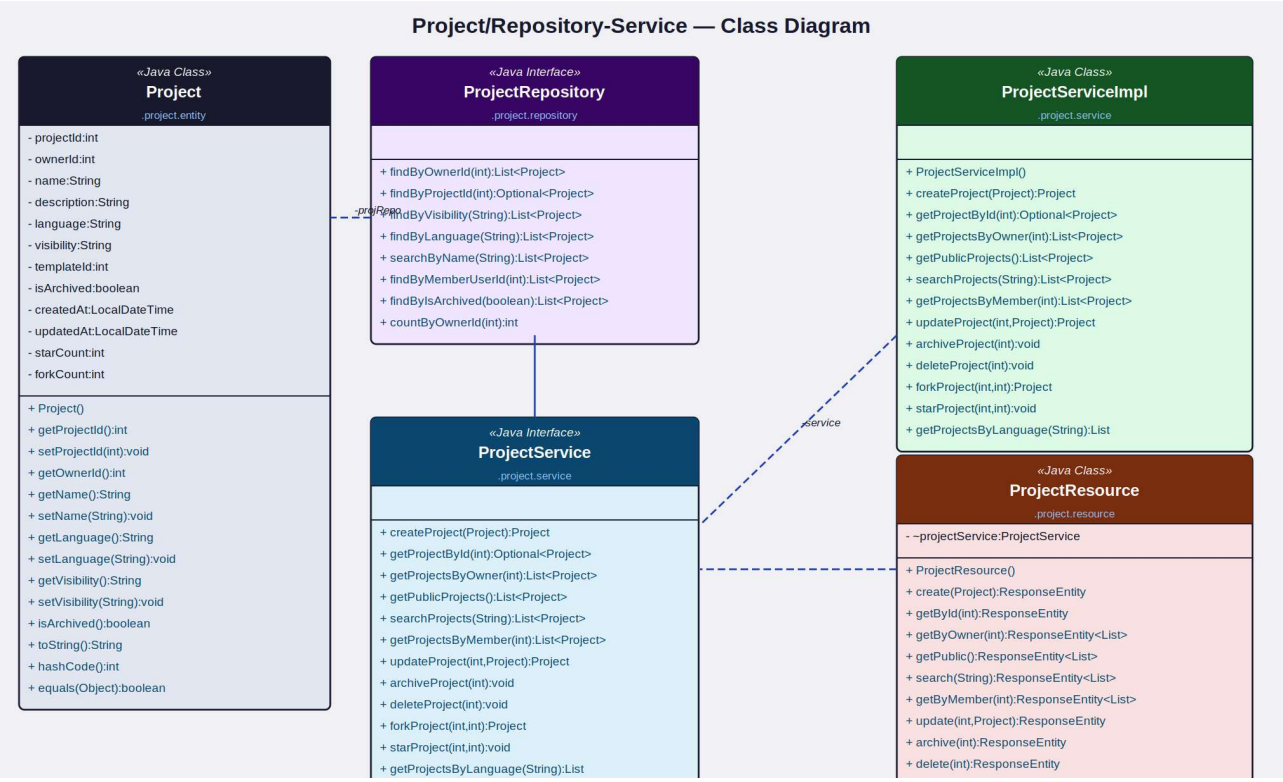

Figure 3: Project/Repository-Service — Class Diagram

Component	Type	Key Attributes / Methods
Project	Java Class (Entity)	projectId, ownerId, name, description, language, visibility (PUBLIC/PRIVATE), templateId, isArchived, createdAt, updatedAt, starCount, forkCount
ProjectRepository	Java Interface	findByOwnerId(), findByProjectId(), findByVisibility(), findByLanguage(), searchByName(), findByMemberUserId(), findByIsArchived(), countByOwnerId()
ProjectServiceImpl	Java Class	createProject(), getProjectById(), getProjectsByOwner(), getPublicProjects(), searchProjects(), getProjectsByMember(), updateProject(), archiveProject(), deleteProject(),

Component	Type	Key Attributes / Methods
		forkProject(), starProject(), getProjectsByLanguage()
ProjectService	Java Interface	Declares all project CRUD, visibility management, search, fork, star, and member-based retrieval operations
ProjectResource	Java Class (REST)	Exposes /projects endpoints: POST (create/fork), GET (by id/owner/member/public/language/search), PUT (update/archive/star), DELETE

4.3 File / Editor-Service

The File/Editor-Service manages the multi-file directory structure of each project. CodeFile records carry a path field (e.g., 'src/main/App.java') enabling nested folder hierarchies. The content field stores the full file text for files; folder entries carry an empty content. The updateFileContent operation records the lastEditedBy user ID and timestamp, providing attribution for collaborative edits. A soft-delete flag preserves deleted files for potential restoration.

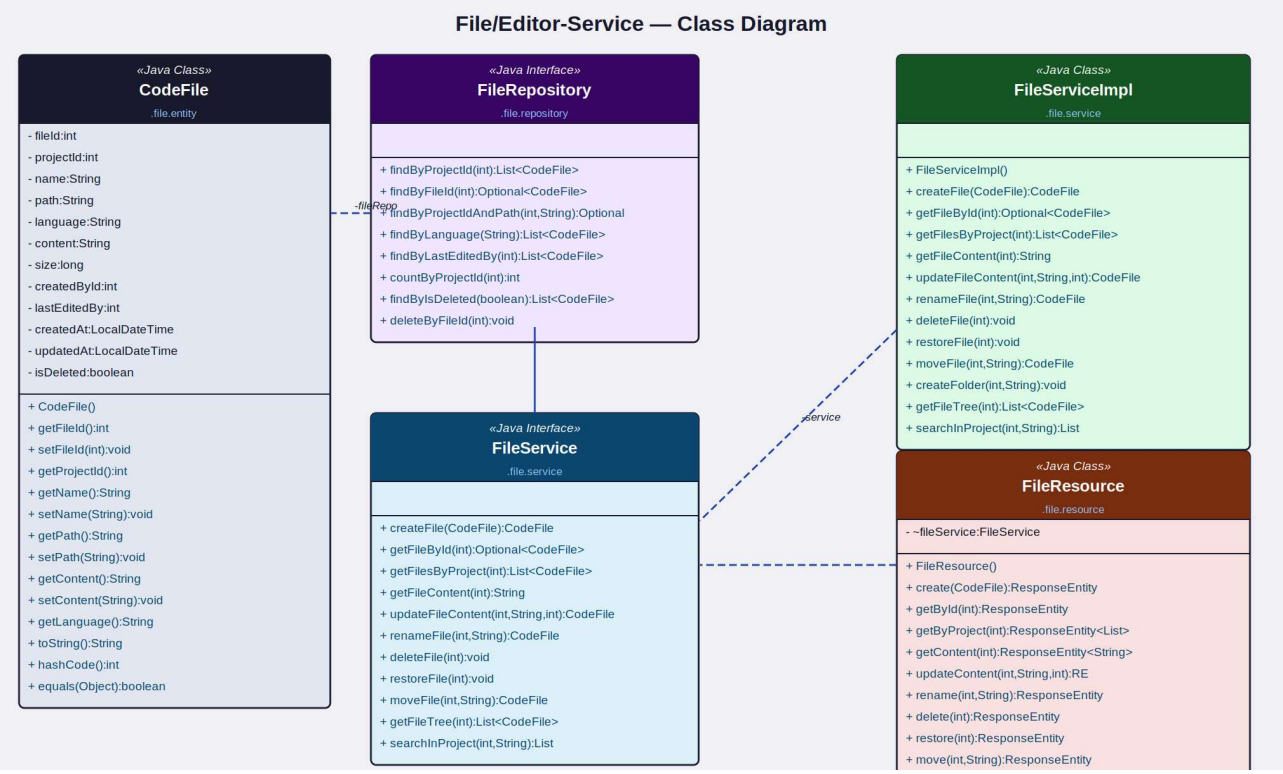

Figure 4: File/Editor-Service — Class Diagram

Component	Type	Key Attributes / Methods
CodeFile	Java Class (Entity)	fileId, projectId, name, path (full directory path), language, content (full text), size (bytes), createdById, lastEditedBy, createdAt, updatedAt, isDeleted
FileRepository	Java Interface	findByProjectId(), findByFileId(), findByProjectIdAndPath(), findByLanguage(), findByLastEditedBy(), countByProjectId(), findByIsDeleted(), deleteByFileId()
FileServiceImpl	Java Class	createFile(), getFileById(), getFilesByProject(), getFileContent(), updateFileContent(), renameFile(), deleteFile(), restoreFile(), moveFile(), createFolder(), getFileTree(), searchInProject()

Component	Type	Key Attributes / Methods
FileService	Java Interface	Declares all file CRUD, content update with attribution, rename/move, soft-delete/restore, folder creation, tree retrieval, and in-project search operations
FileResource	Java Class (REST)	Exposes /files endpoints: POST (create/createFolder), GET (by id/project/content/tree/search), PUT (updateContent/rename/move), DELETE, POST (restore)

4.4 Collaboration / Session-Service

The Collaboration/Session-Service manages real-time co-editing sessions. Each CollabSession is bound to a specific file and tracked by a UUID session ID. The Participant entity records each user who joins, their assigned cursor colour, and their live cursor position (line and column). Session state is distributed via WebSocket (STOMP) — every edit, cursor move, and participant join/leave event is broadcast to all active session members.

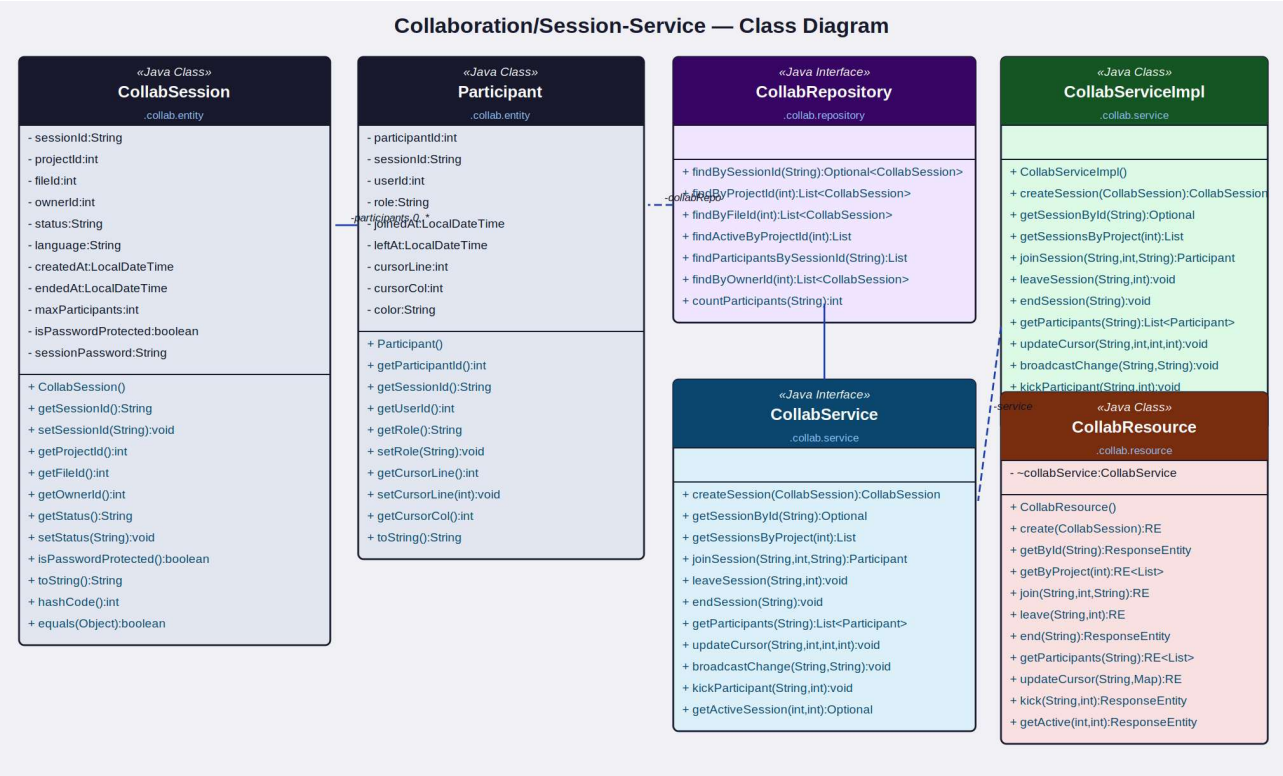

Figure 5: Collaboration/Session-Service — Class Diagram

Component	Type	Key Attributes / Methods
CollabSession	Java Class (Entity)	sessionId (UUID), projectId, fileId, ownerId, status (ACTIVE/ENDED), language, createdAt, endedAt, maxParticipants, isPasswordProtected, sessionPassword
Participant	Java Class (Entity)	participantId, sessionId, userId, role (HOST/EDITOR/VIEWER), joinedAt, leftAt, cursorLine, cursorCol, color (unique per session) — linked to CollabSession (0..*)
CollabRepository	Java Interface	findBySessionId(), findByProjectId(), findByFileId(), findActiveByProjectId(), findParticipantsBySessionId(), findByOwnerId(), countParticipants()

Component	Type	Key Attributes / Methods
CollabServiceImpl	Java Class	createSession(), getSessionById(), getSessionsByProject(), joinSession(), leaveSession(), endSession(), getParticipants(), updateCursor(), broadcastChange(), kickParticipant(), getActiveSession()
CollabService	Java Interface	Declares all session lifecycle, participant management, cursor update, and broadcast operations
CollabResource	Java Class (REST)	Exposes /sessions endpoints: POST (create), GET (by id/project), POST (join/leave/end/kick), PUT (updateCursor), GET (participants/active)

4.5 Version / Snapshot-Service

The Version/Snapshot-Service provides Git-inspired version control for project files. Each Snapshot captures the full file content at a point in time with a SHA-256 hash for integrity, a commit message, a parent snapshot ID forming the history chain, and a branch label. The service supports branch creation, snapshot tagging, diff computation (Myers algorithm), and non-destructive restore (restore creates a new snapshot with the old content).

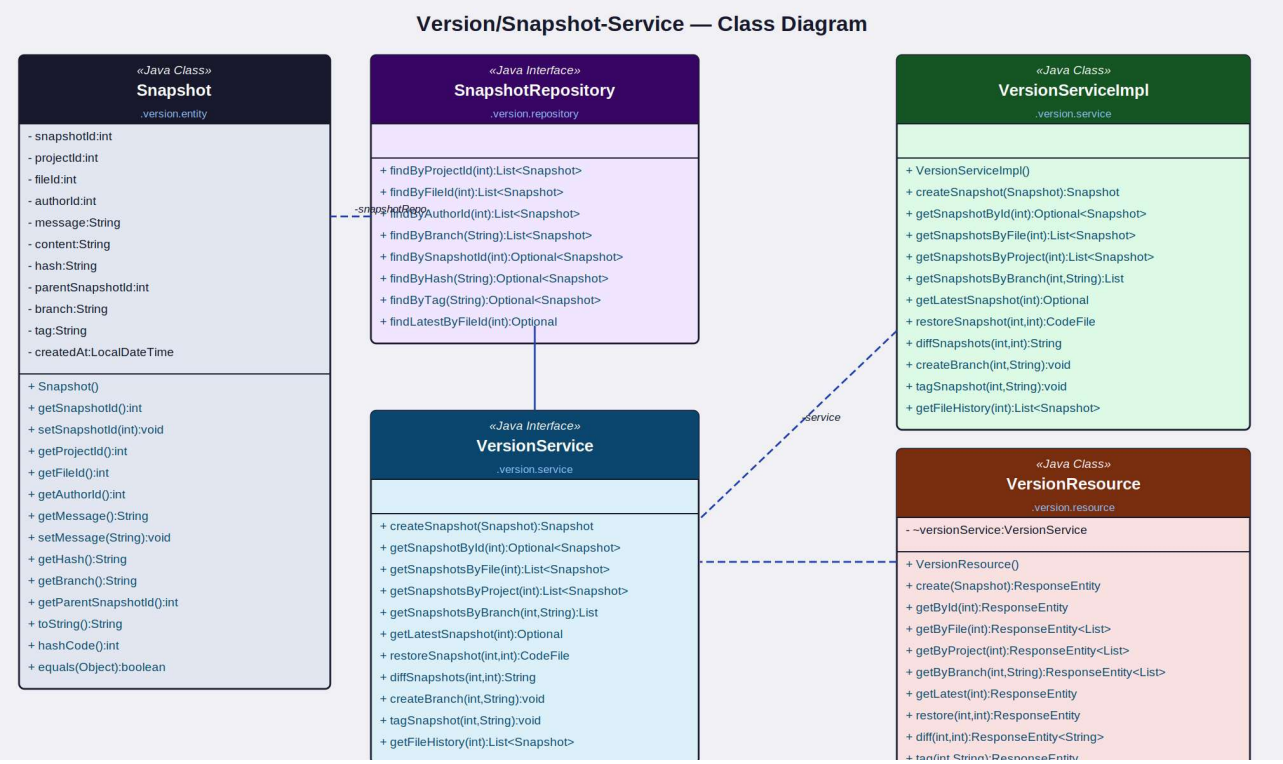

Figure 6: Version/Snapshot-Service — Class Diagram

Component	Type	Key Attributes / Methods
Snapshot	Java Class (Entity)	snapshotId, projectId, fileId, authorId, message (commit message), content (full file text), hash (SHA-256), parentSnapshotId, branch, tag, createdAt
SnapshotRepository	Java Interface	findByProjectId(), findByFileId(), findByAuthorId(), findByBranch(), findBySnapshotId(), findByHash(), findByTag(), findLatestByFileId()
VersionServiceImpl	Java Class	createSnapshot(), getSnapshotById(), getSnapshotsByFile(), getSnapshotsByProject(), getSnapshotsByBranch(), getLatestSnapshot(), restoreSnapshot(), diffSnapshots(), createBranch(), tagSnapshot(), getFileHistory()

Component	Type	Key Attributes / Methods
VersionService	Java Interface	Declares all snapshot CRUD, branch management, tag assignment, diff computation, restore, and history retrieval operations
VersionResource	Java Class (REST)	Exposes /versions endpoints: POST (create/createBranch/tag), GET (by id/file/project/branch/history/latest), POST (restore), GET diff(int,int)

4.6 Code Execution-Service

The Code Execution-Service manages secure code execution in isolated Docker sandbox containers. Each ExecutionJob is submitted asynchronously, queued in RabbitMQ, and processed by a worker pool. The service enforces strict resource limits (CPU, memory, time) per container. stdout streaming is delivered via WebSocket for interactive programs. The service also maintains a registry of supported languages with their sandbox image tags and runtime versions.

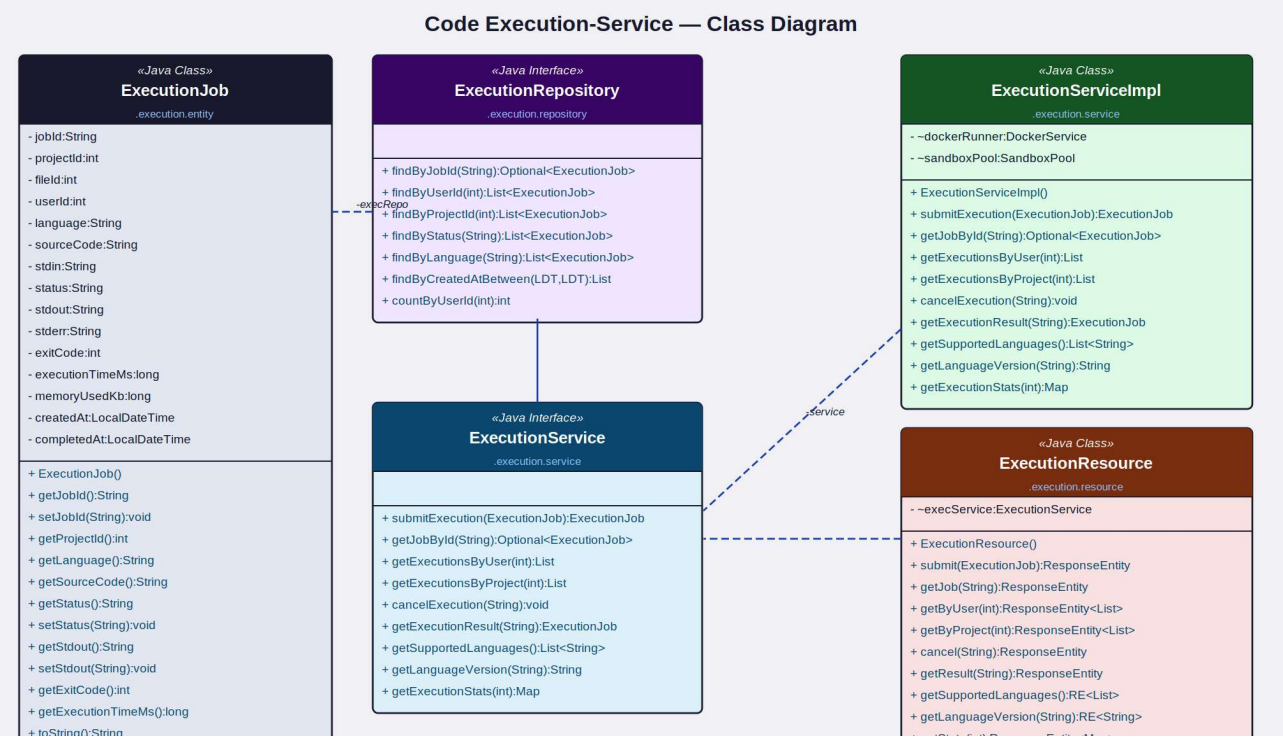

Figure 7: Code Execution-Service — Class Diagram

Component	Type	Key Attributes / Methods
ExecutionJob	Java Class (Entity)	jobId (UUID), projectId, fileId, userId, language, sourceCode, stdin, status (QUEUED/RUNNING/COMPLETED/FAILED/TIMED_OUT/CANCELLED), stdout, stderr, exitCode, executionTimeMs, memoryUsedKb, createdAt, completedAt
ExecutionRepository	Java Interface	findByJobId(), findByUserId(), findByProjectId(), findByStatus(), findByLanguage(), findByCreatedAtBetween(), countByUserId()
ExecutionServiceImpl	Java Class	submitExecution(), getJobById(), getExecutionsByUser(), getExecutionsByProject(), cancelExecution(), getExecutionResult(), getSupportedLanguages(), getLanguageVersion(), getExecutionStats()

Component	Type	Key Attributes / Methods
ExecutionService	Java Interface	Declares code submission, async job tracking, cancellation, result retrieval, language registry, and usage stats operations
ExecutionResource	Java Class (REST)	Exposes /executions endpoints: POST (submit), GET (by jobId/user/project), POST (cancel), GET (result/supportedLanguages/languageVersion/stats)

4.7 Comment / Code-Review-Service

The Comment/Code-Review-Service enables inline code review comments anchored to specific lines of a file. Comments support two-level threading via parentCommentId. The resolved flag enables a review workflow where comments are marked addressed when the underlying code issue is fixed. Comments are linked to a snapshotId so they remain contextually correct even after subsequent file edits. @mention parsing triggers Notification-Service calls.

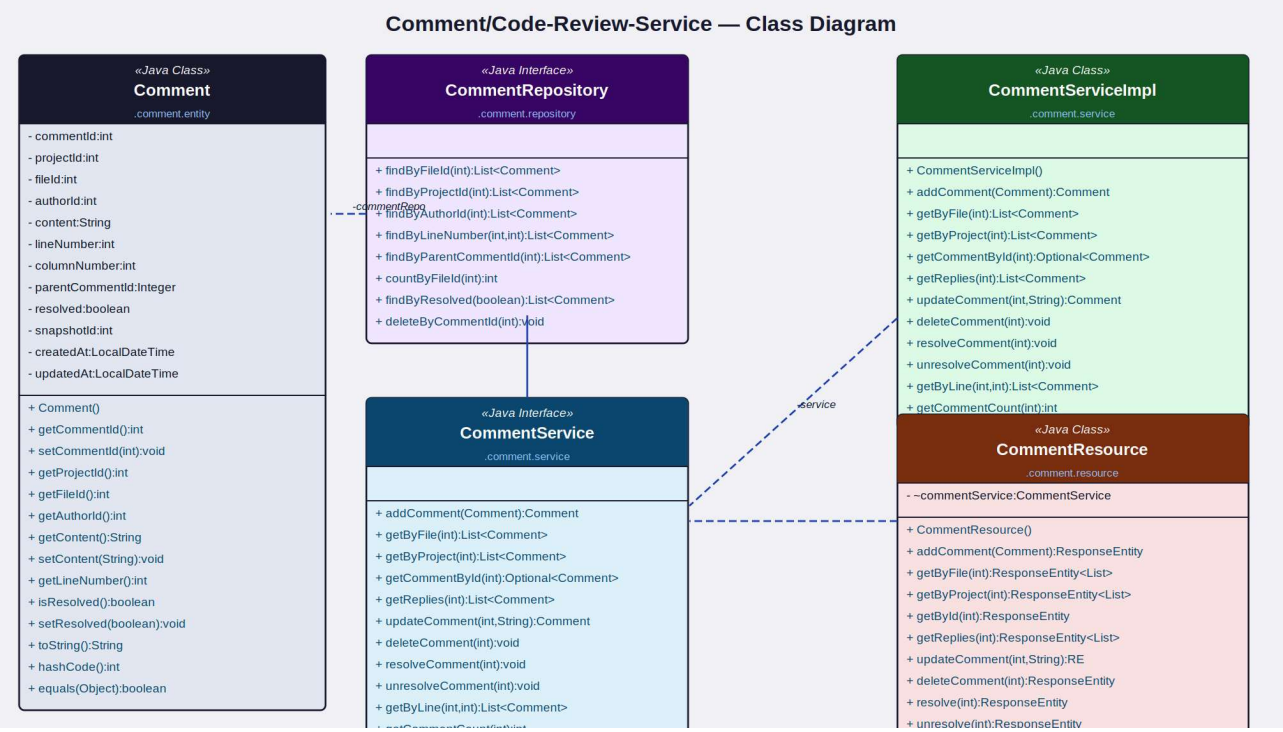

Figure 8: Comment/Code-Review-Service — Class Diagram

Component	Type	Key Attributes / Methods
Comment	Java Class (Entity)	commentId, projectId, fileId, authorId, content, lineNumber, columnNumber, parentCommentId (nullable for top-level), resolved, snapshotId, createdAt, updatedAt
CommentRepository	Java Interface	findByFileId(), findByProjectId(), findByAuthorId(), findByLineNumber(), findByParentCommentId(), countByFileId(), findByResolved(), deleteByCommentId()
CommentServiceImpl	Java Class	addComment(), getByFile(), getByProject(), getCommentById(), getReplies(), updateComment(), deleteComment(), resolveComment(), unresolveComment(), getByLine(), getCommentCount()

Component	Type	Key Attributes / Methods
CommentService	Java Interface	Declares all comment CRUD, reply threading, resolve/unresolve lifecycle, line-based retrieval, and count operations
CommentResource	Java Class (REST)	Exposes /comments endpoints: POST (add), GET (by file/project/id/line), GET replies, PUT (update/resolve/unresolve), DELETE, GET count

4.8 Notification-Service

The Notification-Service dispatches and persists in-app and email alerts for all key collaboration events: session invites, participant joins, comment additions, @mentions, and new snapshots from collaborators. Notifications carry an actorId (who triggered the event), relatedId (the related entity: sessionId, commentId, etc.), and relatedType for deep-linking. Real-time delivery of the unread badge count is pushed to the recipient via WebSocket.

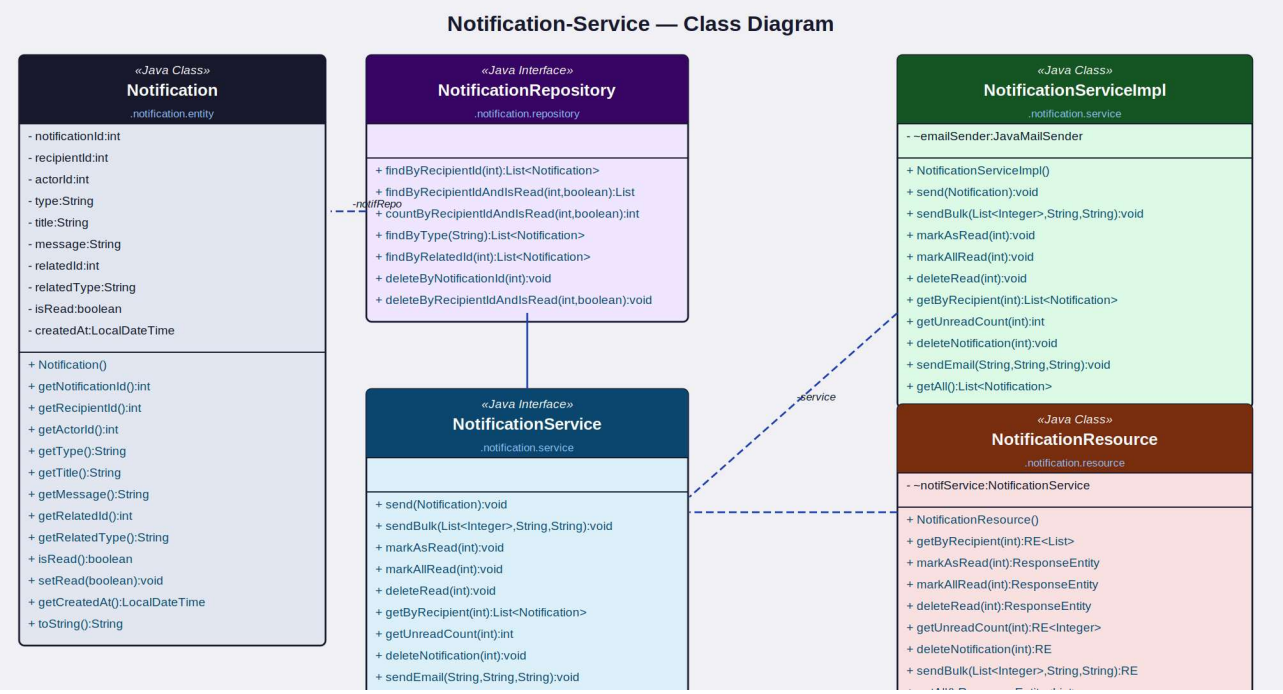

Figure 9: Notification-Service — Class Diagram

Component	Type	Key Attributes / Methods
Notification	Java Class (Entity)	notificationId, recipientId, actorId, type (SESSION_INVITE/COMMENT/MENTION/SNAPSHOT/FORK), title, message, relatedId, relatedType, isRead, createdAt
NotificationRepository	Java Interface	findByRecipientId(), findByRecipientIdAndIsRead(), countByRecipientIdAndIsRead(), findByType(), findByRelatedId(), deleteByNotificationId(), deleteByRecipientIdAndIsRead()
NotificationServiceImpl	Java Class	send(), sendBulk(), markAsRead(), markAllRead(), deleteRead(), getByRecipient(), getUnreadCount(), deleteNotification(), sendEmail(), getAll()
NotificationService	Java Interface	Declares notification dispatch (single/bulk/email), retrieval, read-state management, and deletion operations

Component	Type	Key Attributes / Methods
NotificationResource	Java Class (REST)	Exposes /notifications endpoints: getByRecipient, markAsRead, markAllRead, deleteRead, getUnreadCount, delete, sendBulk, getAll

4.9 Website-Controller (MVC Layer)

The Website-Controller is the Spring MVC integration layer that wires all underlying microservices together and renders views for developers and administrators. It contains three controllers — EditorController, ProjectController, and AdminController — each injecting relevant service dependencies via Spring @Autowired. WebSocket message handlers are co-located in the EditorController for real-time collaboration event routing.

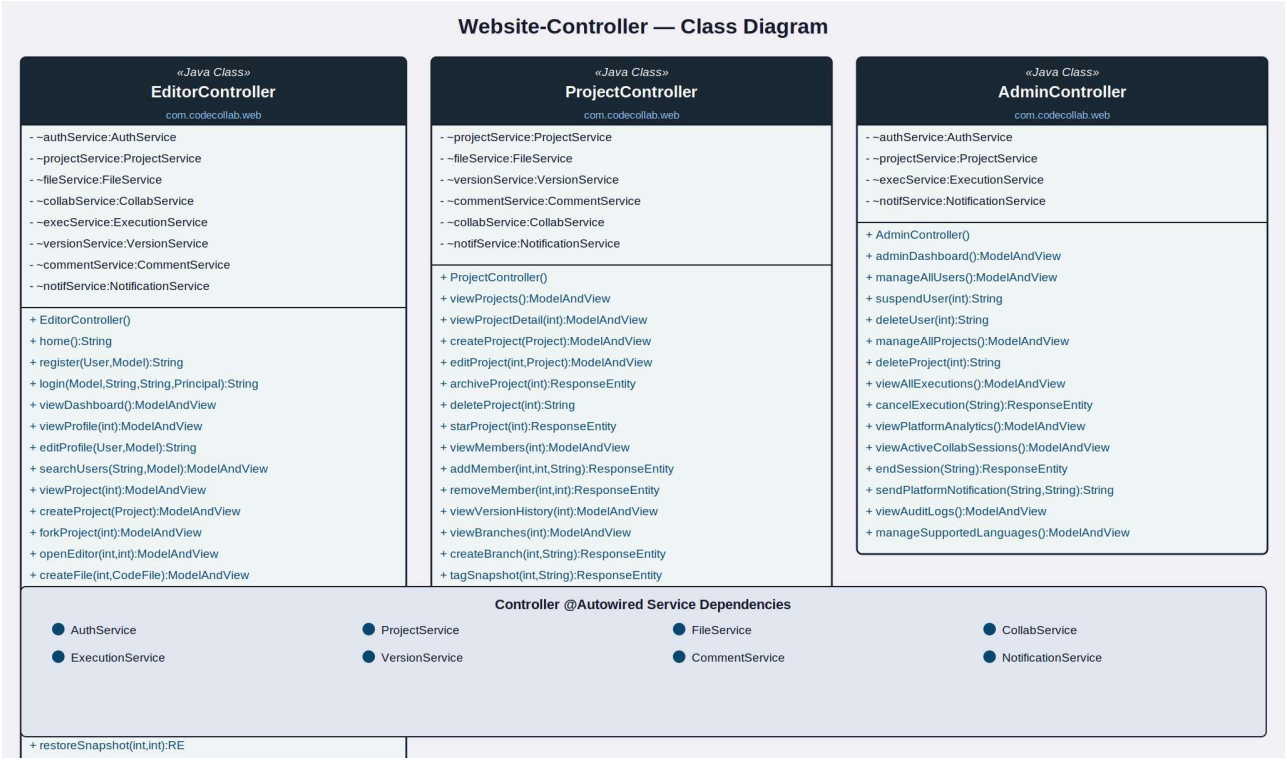

Figure 10: Website-Controller — Class Diagram

Controller	Type	Key Methods
EditorController	Java Class (MVC)	home(), register(), login(), viewDashboard(), viewProfile(), editProfile(), searchUsers(), viewProject(), createProject(), forkProject(), openEditor(), createFile(), deleteFile(), getFileTree(), startSession(), joinSession(), runCode(), getResult(), viewHistory(), restoreSnapshot(), viewNotifications(), logout()
ProjectController	Java Class (MVC)	viewProjects(), viewProjectDetail(), createProject(), editProject(), archiveProject(), deleteProject(), starProject(), viewMembers(), addMember(), removeMember(), viewVersionHistory(), viewBranches(), createBranch(), tagSnapshot(), diffSnapshots(),

Controller	Type	Key Methods
		viewComments(), addComment(), resolveComment(), viewPublicProjects()
AdminController	Java Class (MVC)	adminDashboard(), manageAllUsers(), suspendUser(), deleteUser(), manageAllProjects(), deleteProject(), viewAllExecutions(), cancelExecution(), viewPlatformAnalytics(), viewActiveCollabSessions(), endSession(), sendPlatformNotification(), viewAuditLogs(), manageSupportedLanguages()

5. Microservices Architecture Overview

CodeSync is structured as a set of independently deployable microservices following the EShoppingZone reference pattern. Each service owns its own database schema, REST API contract, service interface, and business logic. Synchronous inter-service calls use RestTemplate; real-time collaboration and execution streaming use WebSocket (STOMP); asynchronous execution jobs are routed via RabbitMQ.

Microservice	Base Package	Primary Domain
auth-service	com.codesync.auth	User accounts, JWT, OAuth2 (GitHub/Google)
project-service	com.codesync.project	Project CRUD, visibility, forking, starring, member access
file-service	com.codesync.file	File/folder CRUD, content management, file tree, in-project search
collab-service	com.codesync.collab	Live session lifecycle, participant management, cursor broadcasting
execution-service	com.codesync.execution	Sandboxed code execution, job tracking, language registry
version-service	com.codesync.version	Snapshot creation, file history, diff, branch, restore, tag
comment-service	com.codesync.comment	Inline code comments, threading, resolve workflow, @mentions
notification-service	com.codesync.notification	In-app and email alerts, bulk dispatch, real-time unread count
codesync-web	com.codesync.web	Spring MVC controllers, Thymeleaf / React view rendering, WebSocket handlers

6. Non-Functional Requirements

Category	Requirement
Performance	Editor load (file content + syntax highlighting) must render within 1 second; WebSocket message round-trip latency under 50ms for collaboration events
Scalability	Microservices independently scalable via Docker/Kubernetes; Redis Pub/Sub distributes WebSocket events across multiple server instances
Security	Passwords stored as bcrypt hashes; JWT tokens expire in 24 hours; OAuth2 for GitHub/Google login; sandboxed code execution with no host-system access; HTTPS enforced
Availability	99.9% uptime SLA; health-check endpoints per service; collaboration sessions degrade gracefully if one participant disconnects
Sandbox Isolation	Each execution runs in an ephemeral Docker container; no network access, no persistent storage, max 10s CPU time, max 256MB RAM, process kill on timeout
Concurrency	Operational Transformation or CRDT guarantees convergent edit consistency under concurrent writes from multiple participants
Collaboration Sync	All cursor movements and edit deltas are broadcast within 100ms to all session participants via WebSocket STOMP channels
Version Integrity	Each snapshot stores a SHA-256 content hash; integrity is verified on restore to detect any corruption
Auto Cleanup	Collaboration sessions idle for 30 minutes are automatically terminated; execution sandbox containers are destroyed within 5 seconds of job completion
Audit Trail	All project member changes, session events, and admin actions are logged with actor, timestamp, and entity reference
Usability	Monaco Editor (VS Code engine) in browser for syntax highlighting, auto-complete, and bracket matching; responsive layout for desktop and large tablet
Maintainability	Services independently deployable; REST API contracts versioned under /api/v1/; Docker image per service with Kubernetes HPA for autoscaling

7. Proposed Technology Stack

Layer	Technology
Backend Framework	Java Spring Boot (Spring MVC, Spring Security, Spring Data JPA, Spring WebSocket)
Authentication	JWT (JSON Web Tokens), Spring Security OAuth2, GitHub OAuth App, Google OAuth2
Database	PostgreSQL (primary relational store per service); Redis (WebSocket session state, pub/sub, execution job queue)
Real-time Collaboration	WebSocket (STOMP) via Spring WebSocket + SockJS; Redis Pub/Sub for multi-instance broadcast
Code Editor (Frontend)	Monaco Editor (VS Code engine) embedded in browser via @monaco-editor/react
Code Execution	Docker Engine API for ephemeral container lifecycle; Judge0 or custom sandbox worker pool
Execution Queue	RabbitMQ for async execution job dispatch; priority queue for fair-use throttling
Version Diff	Myers diff algorithm (java-diff-utils library) for line-by-line snapshot comparison
Frontend	React.js (SPA) with Tailwind CSS; or Thymeleaf + HTMX for server-side rendering
File Storage	PostgreSQL TEXT column for code content (files < 1MB); AWS S3 for large attachment storage
Email	Spring JavaMailSender via AWS SES or SendGrid SMTP for session invite and mention emails
Containerisation	Docker with Docker Compose; Kubernetes for production autoscaling with Horizontal Pod Autoscaler
API Documentation	Swagger / OpenAPI 3.0 for all REST endpoints across all services
CI/CD	GitHub Actions for automated test, build, and deployment pipeline with Docker image push to ECR

8. Glossary

Term	Definition
Project	A top-level container grouping related code files, version history, and collaboration sessions
CodeFile	A single source code file or folder node within a project's directory tree
File Tree	The hierarchical directory structure of a project showing all folders and files
CollabSession	A live co-editing session bound to a specific file where multiple participants edit simultaneously
Participant	A user who has joined an active collaboration session with an assigned cursor colour
Cursor Presence	The real-time display of each participant's cursor position in the shared editor
ExecutionJob	A sandboxed code run submitted to a Docker container, tracking source, output, and performance metrics
Sandbox	An isolated Docker container environment that safely executes user-submitted code with strict resource limits
Snapshot	A saved point-in-time capture of a file's content, analogous to a Git commit
Branch	A named pointer to a chain of snapshots enabling parallel development within a project
Diff	A line-by-line comparison of two file versions highlighting added, removed, and modified lines
SHA-256 Hash	A cryptographic fingerprint of a snapshot's content used to verify data integrity
Operational Transformation	An algorithm that ensures convergent consistency when multiple users edit the same document concurrently
CRDT	Conflict-free Replicated Data Type — an alternative concurrency algorithm that guarantees eventual consistency without conflict resolution
OT	Operational Transformation — see definition above
Fork	Creating a personal copy of another user's public project for independent development
Star	A bookmark mechanism allowing users to save and track projects they find interesting
Inline Comment	A code review annotation attached to a specific line number within a code file
Monaco Editor	The open-source code editor powering VS Code, embedded in the browser for rich editing experience